

JAVA GENERICS

Java: Interfacce e astrazione

Supponiamo di voler definire liste di diversi tipi:

```
interface ListOfNumbers {  
    boolean add(Number elt);  
    Number get(int index);  
}  
interface ListOfIntegers {  
    boolean add(Integer elt);  
    Integer get(int index);  
}
```

... e ListOfStrings e ...

```
// Indispensabile astrarre sui tipi  
interface List<E> {  
    boolean add(E n);  
    E get(int index);  
}
```

Usiamo i tipi!!!

```
List<Number>  
List<Integer>  
List<String>  
List<List<String>>  
...
```

Esempio: una coda

```
class CircleQueue { //coda di cerchi

    // rappresentazione come array
    private Circle[] circles;

    // costruttore
    public CircleQueue(int size) {...}

    // metodi
    public boolean isFull() {...}
    public boolean isEmpty() {...}
    public void enqueue(Circle c) {...}
    public Circle dequeue() {...}
}
```

Esempio: una coda e una seconda coda

```
class CircleQueue { //coda di cerchi

    // rappresentazione come array
    private Circle[] circles;

    // costruttore
    public CircleQueue(int size) {...}

    // metodi
    public boolean isFull() {...}
    public boolean isEmpty() {...}
    public void enqueue(Circle c) {...}
    public Circle dequeue() {...}
}
```

```
class PointQueue { //coda di punti

    // rappresentazione come array
    private Point[] points;

    // costruttore
    public PointQueue(int size) {...}

    // metodi
    public boolean isFull() {...}
    public boolean isEmpty() {...}
    public void enqueue(Point p) {...}
    public Point dequeue() {...}
}
```

Esempio: una coda e una seconda coda

```
class CircleQueue { //coda di cerchi

    // rappresentazione come array
    private Circle[] circles;

    // costruttore
    public CircleQueue(int size) {...}

    // metodi
    public boolean isFull() {...}
    public boolean isEmpty() {...}
    public void enqueue(Circle c) {...}
    public Circle dequeue() {...}
}
```

```
class PointQueue { //coda di punti

    // rappresentazione come array
    private Point[] points;

    // costruttore
    public PointQueue(int size) {...}

    // metodi
    public boolean isFull() {...}
    public boolean isEmpty() {...}
    public void enqueue(Point p) {...}
    public Point dequeue() {...}
}
```

Trova le differenze!

Principio di astrazione

*Each significant piece of functionality in a program should be implemented in just one place in the source code. Where similar functions are carried out by distinct pieces of code, it is generally beneficial to combine them into one by **abstracting out** the varying parts*

Benjamin C. Pierce in Types and Programming Languages (2002)

Esempio: una terza coda (di Object)

```
class CircleQueue { //coda di cerchi

    // rappresentazione come array
    private Circle[] circles;

    // costruttore
    public CircleQueue(int size) {...}

    // metodi
    public boolean isFull()
    public boolean isEmpty()
    public void enqueue(Circle c)
    public Circle dequeue()
}
```

```
class PointQueue { //coda di punti

    // rappresentazione come array
    private Point[] points;

    // costruttore
    public PointQueue(int size) {...}
```

```
class ObjectQueue { //coda di oggetti

    // rappresentazione come array
    private Object[] forme;

    // costruttore
    public ObjectQueue(int size) {...}

    // metodi
    public boolean isFull() {...}
    public boolean isEmpty() {...}
    public void enqueue(Object o) {...}
    public Object dequeue() {...}
}
```

```
    public boolean isFull() {...}
    public boolean isEmpty() {...}
    public void enqueue(Point p) {...}
    public Point dequeue() {...}
```

Usiamo la (terza) coda

```
ObjectQueue cq = new ObjectQueue(10);  
// Crea un'istanza di ObjectQueue di capacità 10  
cq.enqueue(new Circle(new Point(0, 0), 10));  
// Aggiunge alla coda un oggetto Circle con centro (Point) e raggio  
cq.enqueue(new Circle(new Point(1, 1), 5));  
// Aggiunge alla coda un oggetto Circle con centro (Point) e raggio  
...
```


Altre operazioni

Estrarre dalla coda un oggetto istanza di Circle (anche per accedere ai metodi specifici della classe originale) è leggermente più complicato

```
Circle c = cq.dequeue();
```

NO! compilation error: poiché la coda memorizza oggetti di tipo generico **Object**, non possiamo assegnare una espressione di tipo Object a una variabile di tipo Circle, senza una operazione di casting esplicito

```
Circle c = (Circle) cq.dequeue();
```

Tuttavia...

... potrebbe sollevare una `ClassCastException` se la coda contenesse oggetti che non sono di tipo `Circle`

Soluzione:

```
Object o = cq.dequeue();  
    if (o instanceof Circle) {  
        Circle c = (Circle)o;  
    }
```

È una **soluzione complicata**, e che va fatta **a ogni chiamata** a `dequeue`...

A partire da Java 5

L'utilizzo di Object è una pratica meno comune da quando Java ha introdotto i **Generics**, che eliminano la necessità di cast espliciti e migliorano la sicurezza del tipo a tempo di compilazione

Variabile di tipo

```
class Queue<T> {  
    private T[] objects;  
    :  
    public Queue(int size) {...}  
    public boolean isFull() {...}  
    public boolean isEmpty() {...}  
    public void enqueue(T o) {...}  
    public T dequeue() {...}  
}
```

```
Queue<Circle> cq = new Queue<Circle>(10);  
// Coda tipizzata  
cq.enqueue(new Circle(new Point(0, 0), 10));  
cq.enqueue(new Circle(new Point(1, 1), 5));  
Circle c = cq.dequeue();  
// Niente cast!! Controllo statico
```

Da notare

Crea una coda generica che accetta solo oggetti di tipo **Circle** o sue sottoclassi (se esistono): il compilatore si aspetta che tutte le operazioni su questa coda rispettino il tipo specificato



```
Queue<Circle> cq = new Queue<Circle>(10);  
cq.enqueue(new Point(1, 3));
```

- Questo codice genera (giustamente) un **errore a tempo di compilazione**: si sta cercando infatti di aggiungere un oggetto di tipo **Point** a una coda (**Queue<Circle>**) che si è dichiarata contenere solo oggetti di tipo **Circle**
- Dato che **Point** non è un sottotipo di **Circle**, il compilatore rileva un'incompatibilità di tipo

Java Generics

- ▶ **Meccanismo di astrazione** linguistica che consente la definizione di classi e metodi parametrici rispetto al tipo che utilizzano
- ▶ Astrazione che permette di definire **algoritmi che si applicano in contesti diversi**, ma in cui l'interfaccia e il funzionamento generale e l'implementazione possono essere definiti in modo tale da applicarsi indipendentemente dal contesto dell'applicazione

Altro esempio

Definisce un'interfaccia generica che rappresenta una collezione di oggetti di tipo generico **T**

```
interface Collection<T> {  
    boolean contains(T x);  
    boolean remove(T x);  
    boolean add(T x);  
}
```

È una collezione dichiarata per contenere solo oggetti di tipo **String**
Qualsiasi operazione che non rispetti il tipo specificato (**String** in questo caso) genera un errore a **tempo di compilazione**

```
Collection<String> c = ...  
    c.add("hi"); // OK!  
    c.add(2); // static error  
for (String s : c) { // for-each  
    // use s  
}
```

Variabili di tipo

`Set<T>` è un'interfaccia della Java Collections Framework che rappresenta una collezione senza duplicati

Dichiarazione

```
class NewSet<T> implements Set<T> {  
    // non-null elements, contains no duplicates  
    // ...  
    List<T> theRep;  
    T lastItemInserted;  
    ...  
}
```

Lista interna ma che deve essere gestita rispettando le specifiche date

Utilizzo

Dichiarare generici

Sintassi per definire classi e interfacce generiche in Java

- ▶ `class Name<TypeVar1, ..., TypeVarN> {...}`
- ▶ `interface Name<TypeVar1, ..., TypeVarN> {...}`
 - ▶ convenzioni standard
 - T per **T**ype, E per **E**lement,
 - K per **K**ey, V per **V**alue, ...

Variabili di tipo:
`TypeVari`

Istanziare una classe generica significa fornire i **tipi concreti** al momento dell'istanziamento, che andranno a sostituire i parametri generici

`Name<Type1, ..., TypeN>`

Lista Generica

Classe generica che rappresenta una **lista collegata** in cui ogni nodo memorizza un elemento di tipo generico **T**
La classe implementa l'interfaccia generica **Collection<T>**, che definisce operazioni standard per collezioni

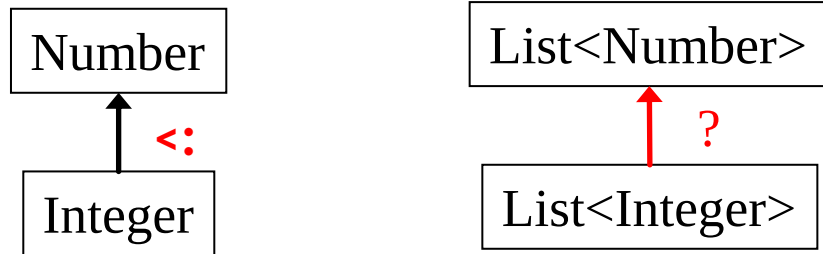
```
class GenList<T> implements Collection<T> {  
    int size = 0;  
    Node<T> head = null; //riferimento al primo Node;  
                          //null se la lista è vuota  
  
    boolean contains(T x) {  
        Node<T> n = head;  
        while (n != null) {  
            if (x.equals(n.data)) return true;  
            n = n.next;  
        }  
        return false;  
    }  
}
```

Lista Generica (Cont)

```
boolean add(T x) {  
    head = new Node<T>(x, head);  
    size++;  
}
```

```
boolean remove(T x) {  
    Node n = head, p = null;  
    while (n != null && !x.equals(n.data)) {p = n; n = n.next; }  
    if (n == null) return false;  
    size--;  
    if (p == null) head = n.next; else p.next = n.next;  
    // elemento in testa  
    return true;  
}
```

Generici e la nozione di sottotipo



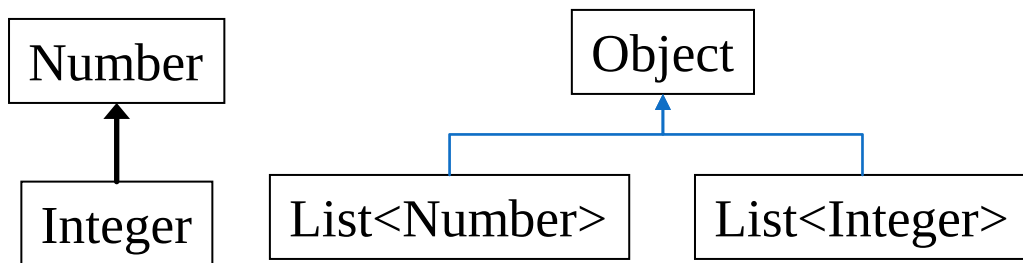
- ▶ **Integer** è un sottotipo di **Number**
- ▶ **List<Integer>** è un sottotipo di **List<Number>**?

Quali sono le regole di Java?

$\text{Type2} <: \text{Type3} \not\Rightarrow \text{Type1} < \text{Type2} > <: \text{Type1} < \text{Type3} >$

Se Type2 e Type3 sono differenti, e Type2 è un sottotipo di Type3 , allora $\text{Type1} < \text{Type2} >$ **non** è un sottotipo di $\text{Type1} < \text{Type3} >$

Formalmente: la nozione di sottotipo usata in Java è **invariante** per le classi generiche



Varianza per tipi

Supponiamo che $A(T)$ sia un opportuno tipo definito usando il Tipo T :

- A è *covariante* se $T <: S$ implica $A(T) <: A(S)$
- A è *contravariante* se $T <: S$ implica $A(S) <: A(T)$
- A è *bivariante* se è sia covariante che contravariante
- A è **invariante** se non è covariante e controvariante

Esempi

Tipi e sottotipi

- ▶ Integer è un sottotipo di Number
- ▶ ArrayList<E> è un sottotipo di List<E> (poiché `ArrayList<E> implements List<E>`)
- ▶ List<E> è un sottotipo di Collection<E> (poiché `List<E> extends Collection<E>`)

Ma

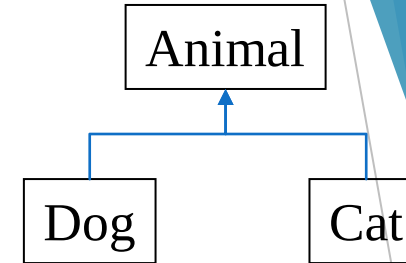
- ▶ List<Integer> **non** è un sottotipo di List<Number>, anche se Integer è un sottotipo di Number

Perché i generici in Java non sono covarianti?

A è *covariante* se $T \leq S$ implica $A(T) \leq A(S)$

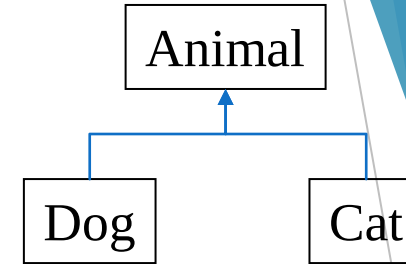
L'invarianza serve per garantire sicurezza del tipo ed evitare errori a run time. Se Java ammettesse la covarianza sui tipi generici, il seguente frammento di codice sarebbe corretto:

```
List<Dog> dogs = new ArrayList<Dog>(); // ArrayList implements List
List<Animal> animals = dogs; // Non consentito poiché non vale covarianza
animals.add(new Cat());
Dog dog = dogs.get(0); // Qual è il tipo effettivo di dog?
```



Perché i generici in Java non sono covarianti?

A è *covariante* se $T \leq S$ implica $A(T) \leq A(S)$



L'invarianza serve per garantire sicurezza del tipo ed evitare errori a run time. Se Java ammettesse la covarianza sui tipi generici, il seguente frammento di codice sarebbe corretto:

può contenere solo oggetti di tipo Dog

```
List<Dog> dogs = new ArrayList<Dog>(); // ArrayList implements List
List<Animal> animals = dogs; // Non consentito poiché non vale covarianza
animals.add(new Cat());
Dog dog = dogs.get(0); // Qual è il tipo effettivo di dog?
```

Dog $\leq$ Animal
ma

Si aspetta solo Dog, ma
potrebbe contenere un Cat

List<Dog> $\nless$ List<Animal>

Il problema è che l'assegnamento crea un *alias* con un tipo diverso (animals e la lista dogs puntano alla stessa struttura dati) e che la lista è *modificabile*!

OCaml ha regole diverse... (covarianza)

```
class c1 = object method m1 = 10 end;;  
class c2 = object method m1 = 5 method m2 = 12 end;;  
let lst1 = [new c1];; (* tipo c1 list *)  
let lst2 = [new c2];; (* tipo c2 list *)  
let lstlst = [lst1; (lst2 :> c1 list)];;  
(* tipo c1 list list *)
```

- È una lista di liste, dove entrambe le sottoliste sono viste come `c1 list`
- La conversione effettua un upcast (covariante)

Funziona!

Ogni oggetto `c2` può essere trattato come un `c1` grazie al suo comportamento compatibile, essendone un sottotipo (possiede il metodo `m1` di `c1`)

Le operazioni consentite su una lista di tipo `c1 list` non violano l'integrità di una lista originariamente di tipo `c2 list` (**immutabilità**)

E con gli array?

- Sappiamo che per i generici la nozione di sottotipo è **invariante**, pertanto, per analogia se **Type1** è un sottotipo di **Type2**, allora **Type1[]** e **Type2[]** **non dovrebbero essere correlati**

E con gli array? La covarianza vale

- ▶ Sappiamo che per i generici la nozione di sottotipo è **invariante**, pertanto, per analogia se **Type1** è un sottotipo di **Type2**, allora **Type1[]** e **Type2[]** **non dovrebbero essere correlati**

- ▶ Ma in questo caso Java ammette la **covarianza**

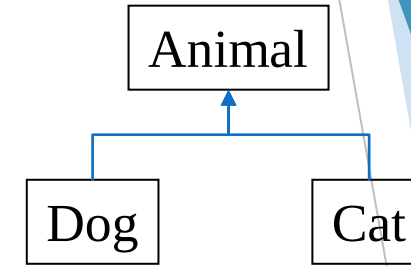
Type2 <: Type3 $\Rightarrow$ Type1[] <: Type1[]

- ▶ **Motivazione storica:**

- ▶ Java (ma anche C#) ha fatto questa scelta prima dell'introduzione dei generici
- ▶ cambiarla dopo sarebbe stato impattante per tutti i programmi Java già esistenti (backward compatibility)

Ripetiamo l'esempio con gli array?

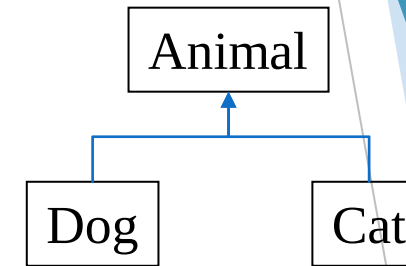
Il seguente esempio **supera i controlli** del compilatore:



```
Dog[] dogs = new Dog[10]; // array di Dog
Animal[] animals = dogs; // Consentito! Poiché per array vale covarianza
animals.add(new Cat());
Dog dog = dogs[0]; // Eccezione a run time: ArrayStoreException
```

Ripetiamo l'esempio con gli array?

Il seguente esempio **supera i controlli** del compilatore:

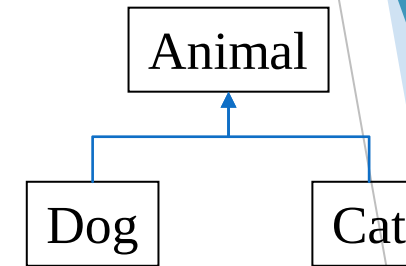


```
Dog[] dogs = new Dog[10]; // array di Dog
Animal[] animals = dogs; // Consentito! Poiché per array vale covarianza
animals.add(new Cat());
Dog dog = dogs[0]; // Eccezione a run time: ArrayStoreException
```

Attenzione: nel caso degli array questo esempio genera un errore rilevato solo a tempo di esecuzione!

Ripetiamo l'esempio con gli array?

Il seguente esempio **supera i controlli** del compilatore:



```
Dog[] dogs = new Dog[10]; // array di Dog
Animal[] animals = dogs; // Consentito! Poiché per array vale covarianza
animals.add(new Dog());
Dog dog = dogs[0]; // Eccezione a run time: ArrayStoreException
```

In questo caso il programma **compila e funziona** correttamente (no eccezione)

Con gli **array** il controllo è **dinamico**: se i **tipi effettivi** (di `dog` e `animals[0]`) sono compatibili tutto funziona, altrimenti viene sollevata un'eccezione

Con i **generici** il controllo, invece, è **statico**: dato che non vale la covarianza, già l'assegnamento `animals = dogs` viene rifiutato a causa dei **tipi statici diversi**, indipendentemente da come potrebbe andare a tempo di esecuzione

Il motivo di questa differenza

- ▶ Già detto: gli array funzionano in questo modo fin dalle prime versioni di Java, mentre i generici sono stati introdotti dopo
 - ▶ **Backward compatibility**: allineare il funzionamento degli array a quello dei generici avrebbe causato problemi ai programmi già esistenti
- ▶ Inoltre, gli array "conoscono" il proprio tipo dinamico (effettivo)
 - Ogni **array** ha nel proprio **descrittore** l'informazione sul tipo dinamico (`Dog[ ]`)
 - La Java Virtual Machine può fare un controllo a run time ed eventualmente sollevare la **ArrayStoreException**, in caso di incompatibilità
 - Con le **classi generiche** invece **l'informazione sul tipo viene eliminata a run time** (processo chiamato **type erasure**, vedremo...), quindi non c'è modo di fare un controllo dinamico: si può ricorrere al solo controllo statico (più vincolante)

Generici e gerarchie

Con i generici niente covarianza... ma c'è un apposito costrutto linguistico per fare qualcosa di simile, che permette di specificare un vincolo sul tipo generico E

Limite superiore gerarchia

```
interface List1<E extends Object> {...}
```

```
interface List2<E extends Number> {...}
```

E deve essere un sottotipo di **Number**

```
List1<Date> // OK, Date è un sottotipo di Object
```

```
List2<Date> // compile-time error:  
            // Date non è sottotipo di Number
```

- **extends** consente di definire vincoli sui tipi generici, introducendo una **flessibilità controllata**
- Questo permette di modellare relazioni gerarchiche di tipo mantenendo il **controllo statico** a tempo di compilazione

Visione effettiva dei generici

Parametri di tipo generico con un vincolo di tipo

```
class Name<TypeVar1 extends Type1,  
    ...,  
    TypeVarN extends TypeN> {...}
```

Limite superiore
gerarchia

Limite superiore
gerarchia

- analogo per le interfacce
- intuizione: `Object` è il limite superiore di default nella gerarchia dei tipi

L'istanziatura resta identica

```
Name<Type1, ..., TypeN>
```

- ▶ Ma si ha *compile-time error* se il tipo non è un **sottotipo** del limite superiore della gerarchia
 - ▶ Attenzione: è sottotipo anche se il limite superiore è un'interfaccia che viene implementata

Usiamo le variabili di tipo

Si possono effettuare tutte le operazioni compatibili con il limite superiore della gerarchia

- concettualmente questo corrisponde a forzare una sorta di preconditione sulla istanziiazione del tipo

```
class List1<E extends Object> {  
    void m(E arg) {  
        arg.asInt( );    // compiler error: E potrebbe  
                          // non avere l'operazione asInt  
    }  
}
```

```
class List2<E extends Number> {  
    void m(E arg) {  
        arg.asInt( );    // OK: Number e tutti i suoi  
                          // sottotipi supportano asInt  
    }  
}
```

Vincoli di tipo

`<TypeVar extends SuperType>`

- *upper bound* va bene il tipo `SuperType` o uno dei suoi sottotipi

`<TypeVar extends ClassA & InterfB & InterfC & ... >`

- *Multiple upper bounds* (al più una classe e arbitrarie interfacce)

`<TypeVar super SubType>`

- *lower bound* va bene il tipo `SubType` o uno dei suoi supertipi

Esempio

```
// insieme ordinato (la classe degli elementi deve
// implementare l'interfaccia Comparable)
public class TreeSet<T extends Comparable<T>> {
    ...
}
```

Altro esempio

Metodo che copia liste (di qualunque tipo)

```
<T> void copyTo(List<T> dst, List<T> src) {  
    for (T t : src)  
        dst.add(t);  
}
```

La soluzione va bene, ma ancora meglio questa (più generica)

```
<T1, T2 extends T1> void copyTo(List<T1> dst, List<T2> src) {  
    for (T2 t : src)  
        dst.add(t);  
}
```

Oppure, analogamente

```
<T1 super T2, T2> void copyTo(List<T1> dst, List<T2> src) {  
    for (T2 t : src)  
        dst.add(t);  
}
```

Altro esempio

Le variabili di tipo possono essere dichiarate anche a livello di singolo metodo!

Metodo che copia liste (di qualunque tipo)

```
<T> void copyTo(List<T> dst, List<T> src) {  
    for (T t : src)  
        dst.add(t);  
}
```

Consente di copiare una lista di Dogs in una lista di Animals (vedi esempi precedenti)

La soluzione va bene, ma ancora meglio questa (più generica)

```
<T1, T2 extends T1> void copyTo(List<T1> dst, List<T2> src) {  
    for (T2 t : src)  
        dst.add(t);  
}
```

Oppure, analogamente

```
<T1 super T2, T2> void copyTo(List<T1> dst, List<T2> src) {  
    for (T2 t : src)  
        dst.add(t);  
}
```

Wildcard: ?

Per introdurre flessibilità mantenendo la sicurezza del tipo, Java supporta wildcard (?)

Wildcard = una variabile di tipo anonima

- ? tipo non conosciuto
- si usano le wildcard quando si usa un tipo esattamente una volta ma non se ne conosce il nome
- l'unica cosa nota è l'unicità del tipo

Sintassi delle wildcard

- ? **extends** **Type**, sottotipo non specificato del tipo **Type**
- ? **super** **Type**, supertipo non specificato del tipo **Type**
- ? notazione semplificata per ? **extends** **Object**

Esempi

```
interface Set<E> {  
    void addAll(Collection<? extends E> c);  
}
```

- è più flessibile rispetto a
void addAll(Collection<E> c);
- zucchero sintattico per
<T extends E> void addAll(Collection<T> c);

Altro esempio:

```
<T> void copyTo(List<T> dst, List<? extends T> src);
```

- zucchero sintattico per
<T1, T2 extends T1> void copyTo(List<T1> dst, List<T2> src);

? vs Object

? Tipo particolare anonimo

```
void printAll(List<?> lst) {...}
```

Quale è la differenza tra `List<?>` e `List<Object>`?

- possiamo istanziare ? con un tipo qualunque: `Object`, `String`, ...
- `List<Object>` è più restrittivo: non si può passare un `List<String>`

Quale è la differenza tra `List<Foo>` e

`List<? extends Foo>`

- nel secondo caso il tipo anonimo è un sottotipo sconosciuto di `Foo`

Too good to be true: type erasure

Tutti i tipi generici e tutti i parametri di tipo vengono rimossi (cancellati, appunto) e sostituiti da `Object` nel processo di compilazione

- motivo: backward-compatibility con il codice vecchio
- morale: a run time, tutte le istanziazioni generiche hanno lo stesso tipo

```
List<String> lst1 = new ArrayList<String>( );  
List<Integer> lst2 = new ArrayList<Integer>( );  
lst1.getClass( ) == lst2.getClass( ) // true
```

`getClass()` restituisce il tipo effettivo

Quindi il compilatore usa i Generics per fare un type checking più accurato: poi scarta l'informazione che nel bytecode non appare più

Esempio

Il compilatore usa i generici per i controlli statici e poi li elimina dal codice (non esistono nel bytecode):

```
class Vector<T> {  
    T[] v; int sz;  
    Vector() {  
        v = new T[15];  
        sz = 0;  
    }  
    <U extends Comparator<T>>  
    void sort(U c) {  
        ...  
        c.compare(v[i], v[j]);  
        ...  
    }  
}  
...  
Vector<Button> v;  
v.addElement(new Button());  
Button b = v.get(0);
```

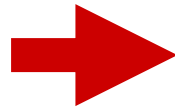


```
class Vector {  
    Object[] v; int sz;  
    Vector() {  
        v = new Object[15];  
        sz = 0;  
    }  
    void sort(Comparator c) {  
        ...  
        c.compare(v[i], v[j]);  
        ...  
    }  
}  
...  
Vector v;  
v.addElement(new Button());  
Button b = (Button)b.get(0);
```

Esempio

Il compilatore usa i generici per i controlli statici e poi li elimina dal codice (non esistono nel bytecode):

```
class Vector<T> {  
    T[] v; int sz;  
    Vector() {  
        v = new T[15];  
        sz = 0;  
    }  
    <U extends Comparator<T>>  
    void sort(U c) {  
        ...  
        c.compare(v[i], v[j]);  
        ...  
    }  
}  
...  
Vector<Button> v;  
v.addElement(new Button());  
Button b = v.get(0);
```



```
class Vector {  
    Object[] v; int sz;  
    Vector() {  
        v = new Object[15];  
        sz = 0;  
    }  
    void sort(Comparator c) {  
        ...  
        c.compare(v[i], v[j]);  
        ...  
    }  
}  
...  
Vector v;  
v.addElement(new Button());  
Button b = (Button)b.get(0);
```

Generici e casting

```
List<?> lg = new ArrayList<String>( ); // ok  
List<String> ls = (List<String>) lg; // warning
```

Nella documentazione Java si scrive: “*Compiler gives an unchecked warning, since this is something the run-time system **will not check for you***”

- ▶ A causa della **type erasure**, l'informazione sul tipo generico (**String** in questo caso) viene eliminata durante la compilazione
- ▶ A run time l'interprete può solo controllare che **lg** sia una lista (**List**) e non che sia una lista di stringhe (**List<String>**)
- ▶ Se il cast fosse errato, si potrebbe verificare un **ClassCastException** solo nel momento in cui si accede agli elementi della lista

Generici e casting

```
import java.util.ArrayList;

public class Prova<E extends Number> {
    public static void main(String[] args) {
        ArrayList<String> a = new ArrayList<String>();
        a.add("ciao");
        ArrayList<Integer> b = magic(a); // fa magicamente il cast...
        b.add(100);
        System.out.println(b.get(0)); // stampa "ciao"
        System.out.println(b.get(1)); // stampa 100
    }

    public static ArrayList<Integer> magic(ArrayList<?> a) {
        return (ArrayList<Integer>) a;
    }
}
```

Generici e casting

```
import java.util.ArrayList;

public class Prova<E extends Number> {
    public static void main(String[] args) {
        ArrayList<String> a = new ArrayList<String>();
        a.add("ciao");
        ArrayList<Integer> b = magic(a); // fa magicamente cast...
        b.add(100);
        System.out.println(b.get(0)); // stampa "ciao"
        System.out.println(b.get(1)); // stampa 100
    }

    public static ArrayList<Integer> magic(ArrayList<?> a) {
        return (ArrayList<Integer>) a;
    }
}
```

Il fatto che b contenga elementi di tipo diverso mostra che in realtà è di tipo `ArrayList<Object>` (type erasure)

Il compilatore segnala un **warning** (unchecked cast)

Java Generics: vantaggi

- ▶ I Generics consentono di specificare i tipi di dati che una classe, un'interfaccia o un metodo possono utilizzare
- ▶ È chiaro quale tipo di dati una collezione o un metodo deve manipolare
- ▶ Questo permette di rilevare errori di tipo durante la compilazione, anziché a run time e migliora la leggibilità del codice
- ▶ I Generics permettono di scrivere classi e metodi riutilizzabili che possono lavorare con diversi tipi di dati, senza duplicare il codice per ciascun tipo
- ▶ Le collezioni Java (come ArrayList, HashMap, Set) e molte librerie standard si basano sui Generics
- ▶ I Generics sono fondamentali per implementare strutture dati generiche e algoritmi flessibili, come stack, code o alberi

Java Generics (JG), considerazioni finali

- ▶ I JG aiutano a migliorare il polimorfismo della soluzione
- ▶ Il compilatore verifica l'utilizzo corretto dei generici
- ▶ I parametri di tipo vengono eliminati durante la compilazione e il "class file" che ne risulta è un normale class file, privo di poliformismo parametrico
- ▶ Limite principale: il tipo effettivo viene perso a run time a causa della type erasure